



## CAPÍTULO 3 — REDES NEURAIS ARTIFICIAIS

# O Perceptron

*Fundamentos, geometria e algoritmo de aprendizado do primeiro neurônio artificial treinável — com aprofundamento matemático e comparação com a Adaline*

Material de referência: Redes Neurais para Engenheiros e Cientistas da Computação — Cap. 3 (Perceptron) e material de aprofundamento em áudio sobre Perceptron e Adaline

Aula preparada para a disciplina de Inteligência Artificial

# O que vamos cobrir

## 01 Introdução e contexto histórico

Rosenblatt, a inspiração biológica e a arquitetura de camada única

## 02 Funcionamento matemático

Entradas, pesos, limiar, potencial de ativação e funções de ativação

## 03 Interpretação geométrica

Fronteira de decisão, reta, plano, hiperplano e o papel de  $w$  e  $\theta$

## 04 Convergência e limitações

Teorema de Minsky & Papert e o problema do XOR

## 05 Algoritmo de treinamento

Regra de Hebb, equações de ajuste, pseudocódigo e conceito de época

## 06 Perceptron $\times$ Adaline

Gradiente descendente, EQM e a Regra Delta (LMS)

## 07 Projeto prático e exercícios

Aplicação em classificação de óleo e exercícios propostos

# 01

## Introdução e Contexto Histórico

*De onde vem o Perceptron e por que ele importa até hoje*

# O nascimento do Perceptron

- **Idealizado por Frank Rosenblatt em 1958.**
- Forma mais simples de configuração de uma rede neural artificial.
- Propósito original: modelo computacional inspirado na retina humana.
- Objetivo: percepção eletrônica de sinais e identificação de padrões geométricos (letras, números).

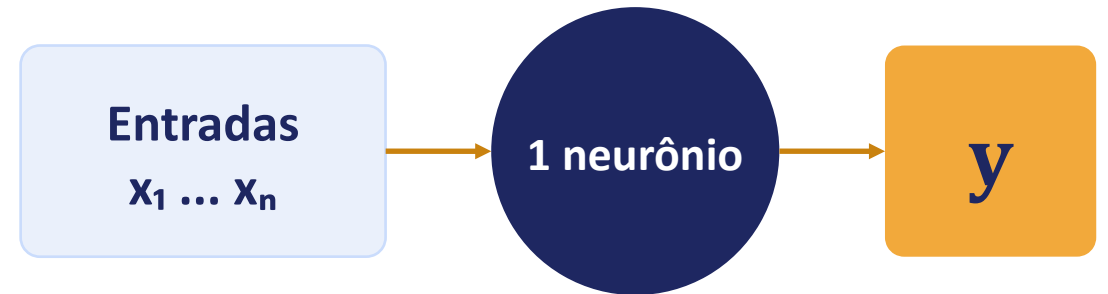


## Concepção física original

- Fotocélulas captam sinais elétricos, simulando a retina.
- Sinais são ponderados por resistores sintonizáveis, ajustáveis durante o treinamento.
- Um elemento somador combina os sinais ponderados para reconhecer padrões.

# Arquitetura: simplicidade estrutural

- **Constituído por apenas uma camada neural.**
- Contém somente um neurônio artificial.
- Arquitetura feedforward (alimentação direta) de camada única.
- Fluxo de informação: entrada → saída, sem qualquer realimentação (feedback).
- Mesmo simples, atraiu enorme atenção da comunidade científica e da IA.



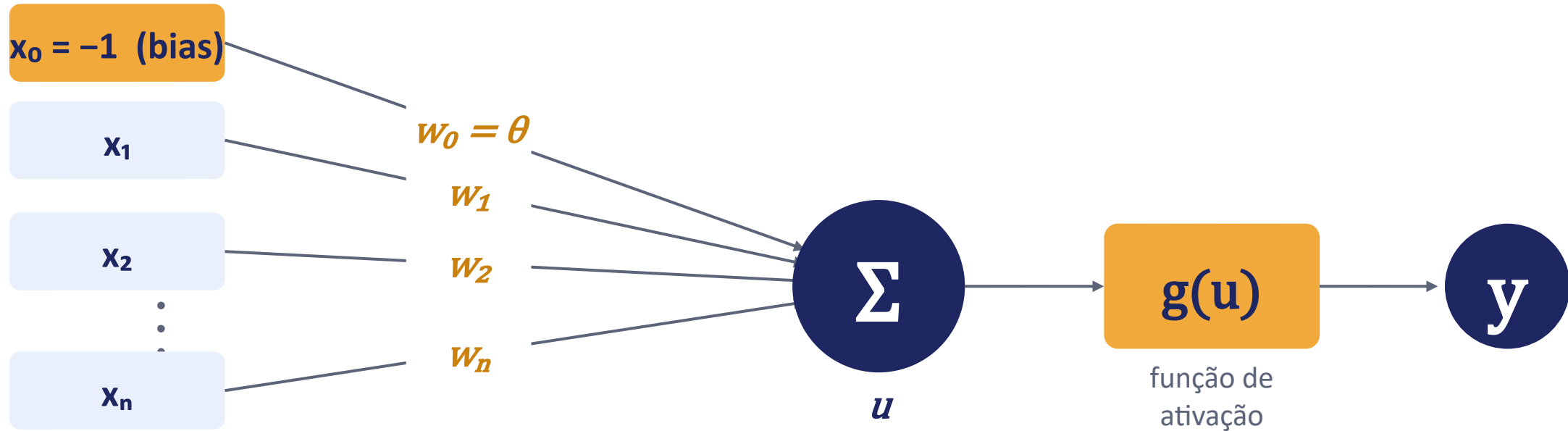
*Sem realimentação: a informação nunca volta para trás*

# 02

## Funcionamento Matemático

*Como o neurônio único do Perceptron processa um sinal*

# Diagrama do neurônio de McCulloch-Pitts / Perceptron



*cada entrada  $x_i$  é ponderada por um peso sináptico  $w_i$  antes de chegar ao somador*

# Equações do neurônio

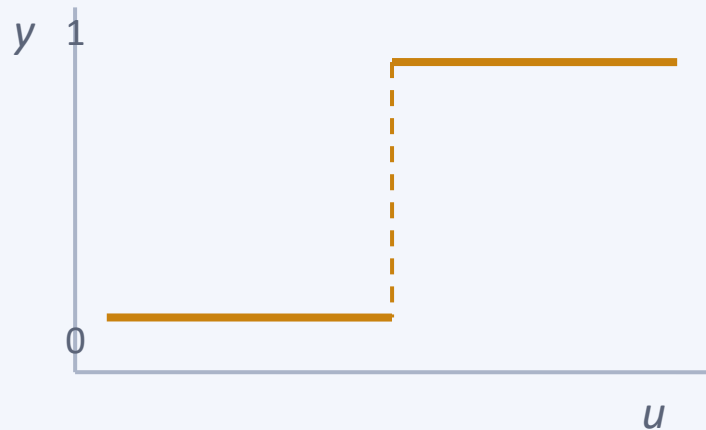
$$u = \sum_{i=1}^n w_i \cdot x_i - \theta$$
$$y = g(u)$$

- $x_i$  — sinais de entrada (informações do processo mapeado)
- $w_i$  — pesos sinápticos, quantificam a importância de cada entrada
- $\theta$  — limiar de ativação (threshold)
- $u$  — potencial de ativação (combinador linear)
- $g(\cdot)$  — função de ativação, gera a saída  $y$  do neurônio

# Funções de ativação do Perceptron

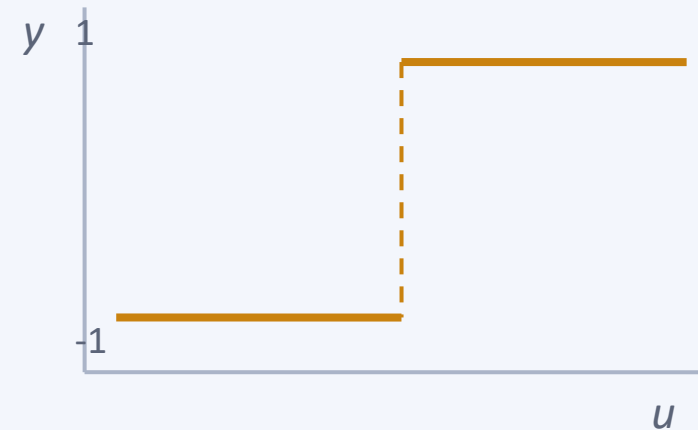
*As entradas  $x_i$  podem ser reais ou binárias. Na prática, normalizar as entradas melhora o desempenho do treinamento.*

## Função degrau



$$y = 1, \text{ se } u \geq 0; \quad y = 0, \text{ se } u < 0$$

## Função degrau bipolar



$$y = 1, \text{ se } u \geq 0; \quad y = -1, \text{ se } u < 0$$

# O limiar como um peso adicional

- Por razões de implementação computacional,  $\theta$  pode ser tratado como um peso sináptico adicional  $w_0$ .
- Associa-se  $w_0$  a uma entrada fixa unitária negativa:  $x_0 = -1$ .
- Vantagem: evita uma instrução condicional extra só para somar o limiar — reduz tudo a um único produto escalar.

$$u = X^T W = w_0 x_0 + w_1 x_1 + \dots + w_n x_n = \sum_{i=0}^n w_i x_i$$

## Antes → Depois

$$u = w_1 x_1 + w_2 x_2 - \theta$$

↓ incorporando  $\vartheta = w_0 \cdot x_0$

$$u = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$W = [\theta \ w_1 \ w_2 \ \dots \ w_n]^T$$

$$X^k = [-1 \ x_1^k \ x_2^k \ \dots \ x_n^k]^T$$

Assim,  $u$  vira um simples produto escalar  $w^T x$  — mesma operação vetorizada em qualquer linguagem de programação.

TABELA 3.1

# Resumo dos parâmetros característicos

| Parâmetro                     | Característica                   |
|-------------------------------|----------------------------------|
| Entradas ( $x_i$ )            | Reais ou binárias                |
| Pesos sinápticos ( $w_i$ )    | Reais (iniciados aleatoriamente) |
| Limiar ( $\theta$ )           | Real (iniciado aleatoriamente)   |
| Saída ( $y$ )                 | Binária                          |
| Função de ativação $g(\cdot)$ | Degrau ou degrau bipolar         |
| Processo de treinamento       | Supervisionado                   |
| Regra de aprendizado          | Regra de Hebb                    |

# 03

## Interpretação Geométrica

*O Perceptron como discriminador linear*

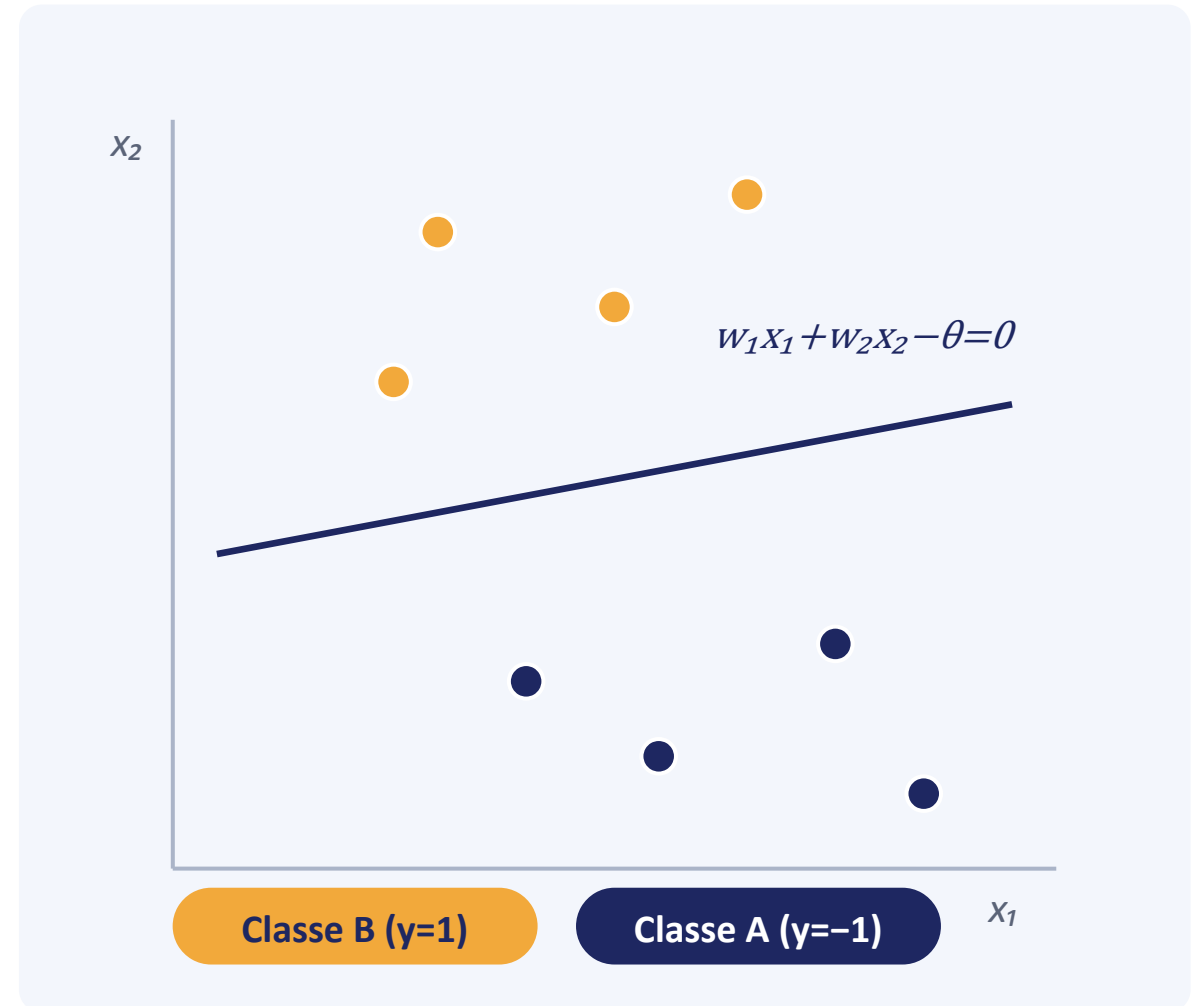
# O Perceptron como discriminador linear

$$y = \begin{cases} 1, & \text{se } w_1x_1 + w_2x_2 - \theta \geq 0 \\ -1, & \text{se } w_1x_1 + w_2x_2 - \theta < 0 \end{cases}$$

A fronteira de decisão é a reta:

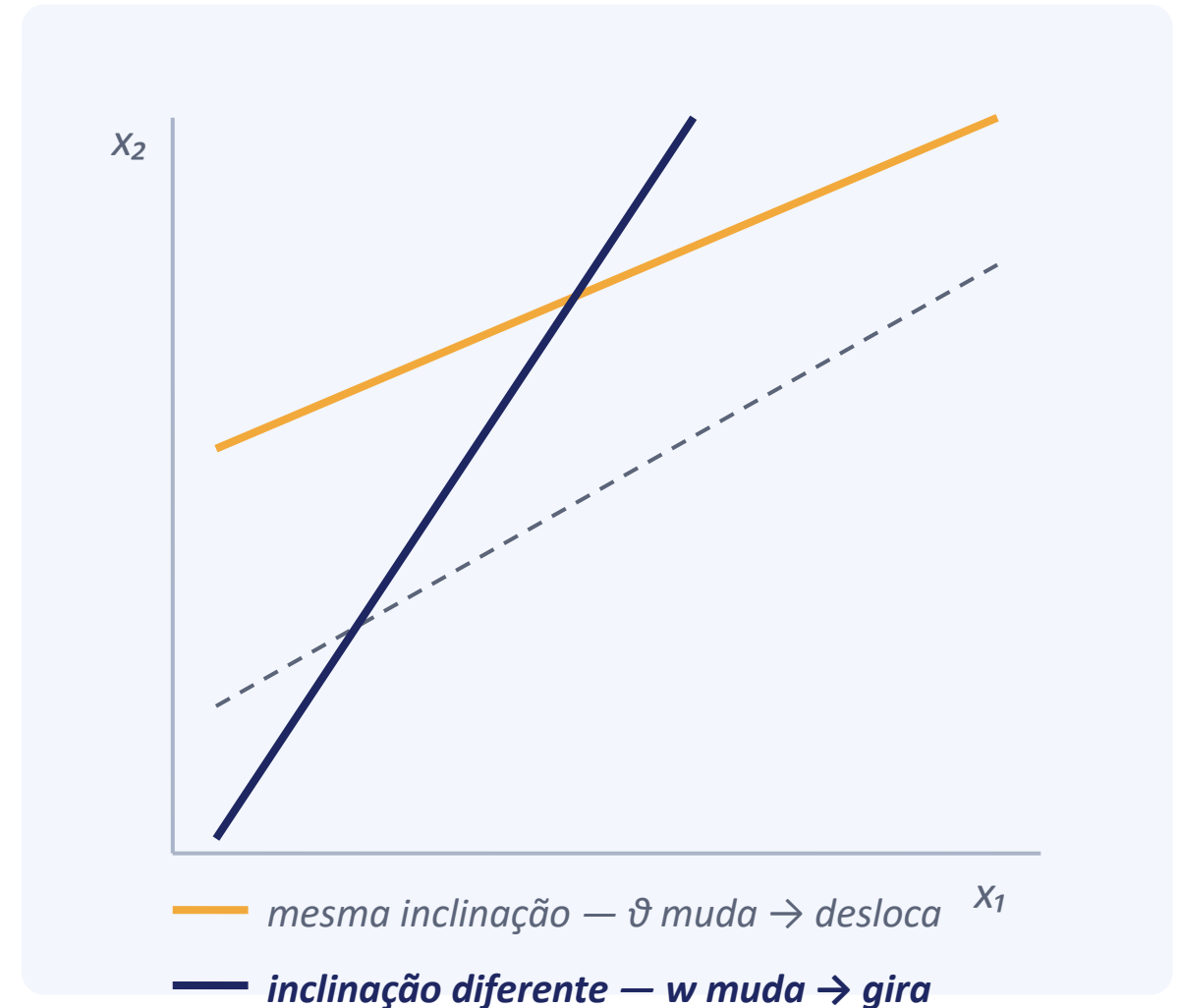
$$w_1x_1 + w_2x_2 - \theta = 0$$

- $y = -1 \rightarrow$  padrões (Classe A) abaixo da reta
- $y = 1 \rightarrow$  padrões (Classe B) acima da reta



# O papel de $w$ e de $\theta$ na reta

- Os pesos  $w_1, w_2, \dots, w_n$  definem o coeficiente angular (inclinação) do hiperplano.
- O limiar  $\theta$  (ou  $w_0$ ) define o coeficiente linear — a posição onde a reta intercepta os eixos.
- **Mudar  $w$  gira a reta; mudar  $\theta$  desloca a reta paralelamente.**



## Caso especial: limiar nulo ( $\theta = 0$ )

Se o limiar  $\theta$  for assumido igual a zero, a reta de separação obrigatoriamente passará pela origem do sistema de coordenadas.

*Implicação prática: com  $\vartheta = 0$ , o Perceptron perde a capacidade de deslocar a fronteira — só consegue representar separações que passam pelo ponto de origem, limitando fortemente os problemas que pode resolver.*



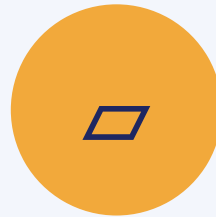
# Generalizando as dimensões



**2D**

**Reta**

duas entradas ( $x_1, x_2$ )



**3D**

**Plano**

três entradas ( $x_1, x_2, x_3$ )



**n-D**

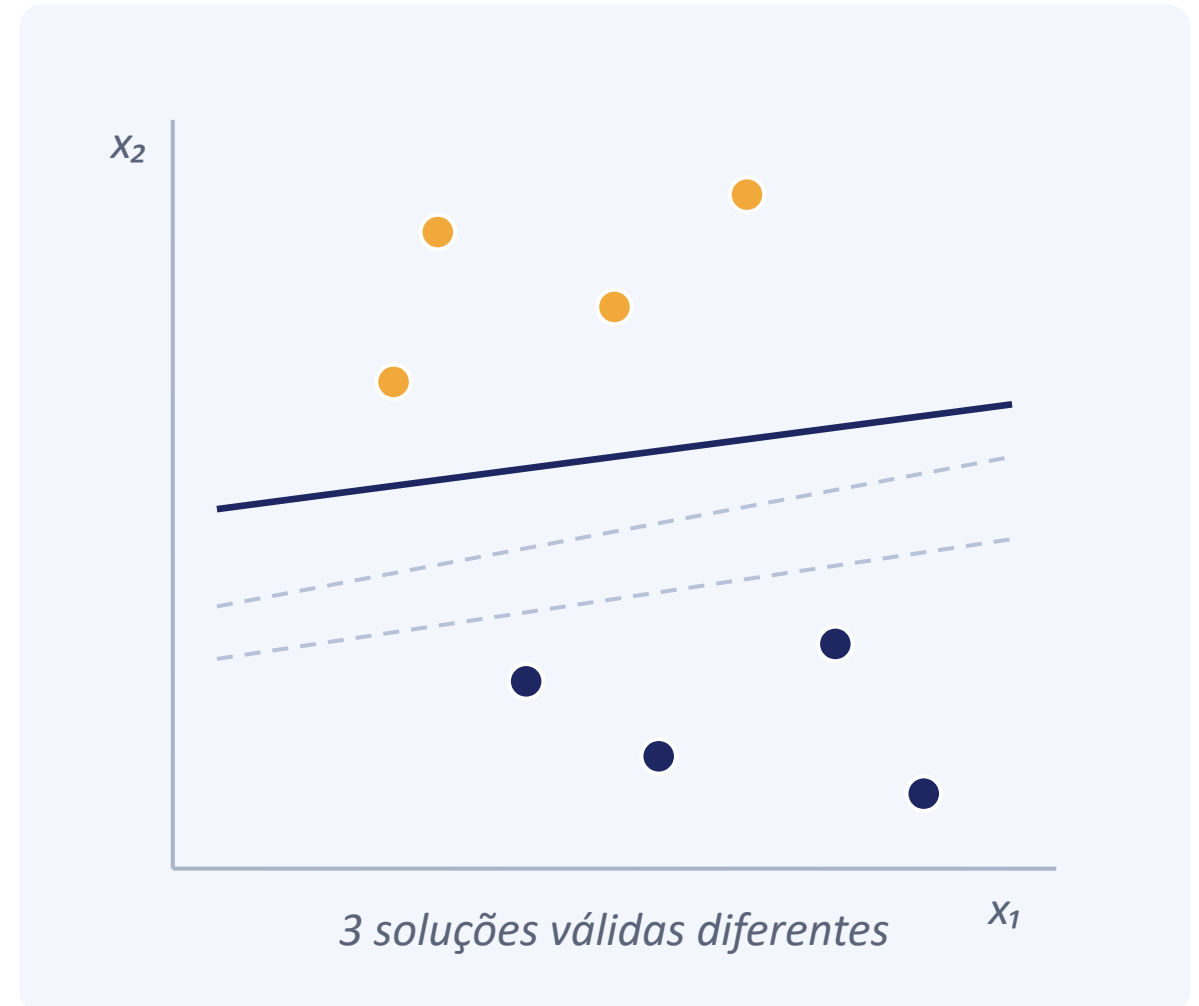
**Hiperplano**

quatro ou mais entradas

*A fronteira de decisão do Perceptron é sempre linear — daí o nome discriminador linear.*

# A solução do Perceptron não é única

- Para qualquer problema linearmente separável, existem infinitas retas (ou hiperplanos) válidas capazes de separar as classes.
- A posição final obtida depende diretamente dos pesos sorteados na inicialização aleatória.
- **Diferentes projetistas treinando o mesmo problema podem obter retas diferentes — todas corretas.**



# 04

## Convergência e Limitações

*Quando o Perceptron funciona — e quando ele falha*

# Teorema da Convergência do Perceptron

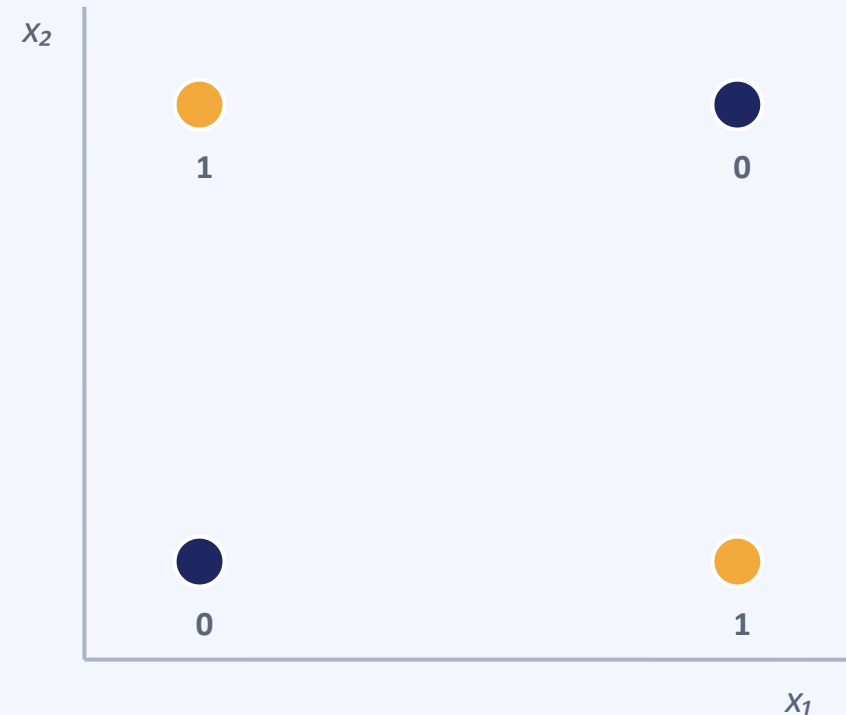
**A condição indispensável para que o Perceptron de camada simples consiga classificar padrões é que as classes do problema sejam LINEARMENTE SEPARÁVEIS.**

- Se uma única reta (ou hiperplano) consegue dividir totalmente os dados em duas classes → o algoritmo converge em um número finito de épocas.
- Se as classes não forem linearmente separáveis → a rede não converge (entra em oscilação/laço permanente).

# A limitação histórica: o problema do XOR

- O Perceptron de camada única é incapaz de resolver problemas cujos padrões não sejam linearmente separáveis.
- Exemplo clássico: a função lógica OU-Exclusivo (XOR).
- Não existe nenhuma reta capaz de separar as saídas ativas das inativas no plano cartesiano.
- **Essa limitação teórica desacelerou as pesquisas em IA até o surgimento das redes multicamadas, nos anos 1980.**

**Tabela-verdade do XOR**



*✗ nenhuma reta separa ● de ●*

# 05

## Treinamento do Perceptron

*Como a rede aprende a ajustar seus pesos*

# Treinamento supervisionado e a Regra de Hebb

O ajuste dos pesos sinápticos e do limiar ocorre por meio de um treinamento SUPERVISIONADO fundamentado na regra de aprendizado de Hebb.



## Condição excitatória

Se a saída produzida for DIFERENTE da saída desejada, os pesos e o limiar são ajustados proporcionalmente aos valores das entradas.



## Condição inibitória

Se a saída produzida for IGUAL à desejada, os pesos e o limiar permanecem inalterados.

*Repete-se sequencialmente para todas as amostras até que não haja mais erros.*

## Equações de ajuste

$$w_i(\text{atual}) = w_i(\text{anterior}) + \eta \cdot (d^k - y) \cdot x_i^k$$

*Pesos sinápticos*

$$\theta(\text{atual}) = \theta(\text{anterior}) + \eta \cdot (d^k - y) \cdot (-1)$$

*Limiar de ativação*

- $d^k$  — saída desejada para a amostra  $k$
- $y$  — saída produzida pela rede
- $\eta$  — taxa de aprendizagem ( $0 < \eta < 1$ )

# Representação vetorial

Incorporando o limiar dentro do vetor de pesos e estendendo o vetor de entradas com  $x_0 = -1$ :

$$w = [\theta \ w_1 \ w_2 \ \dots \ w_n]^T$$

$$x^k = [-1 \ x_1^k \ x_2^k \ \dots \ x_n^k]^T$$

A regra de atualização em notação vetorial / algorítmica:

$$w \leftarrow w + \eta \cdot (d^k - y) \cdot x^k$$

# Algoritmo de treinamento — inicialização

- 1 Obter o conjunto de amostras de treinamento  $\{x^{(k)}\}$ .
- 2 Associar a saída desejada  $\{d^{(k)}\}$  para cada amostra.
- 3 Iniciar o vetor  $w$  com valores aleatórios pequenos.
- 4 Especificar a taxa de aprendizagem  $\eta$ .
- 5 Iniciar o contador de número de épocas ( $\text{época} \leftarrow 0$ ).

# Algoritmo de treinamento — laço principal

```
Repetir:  
    erro ← "inexistente"  
    Para todas as amostras  $(x^{(k)}, d^{(k)})$  :  
         $u \leftarrow w^T \cdot x^{(k)}$   
         $y \leftarrow \text{sinal}(u)$   
        Se  $y \neq d^{(k)}$  então:  
             $w \leftarrow w + \eta \cdot (d^{(k)} - y) \cdot x^{(k)}$   
            erro ← "existente"  
    época ← época + 1  
Até que: erro = "inexistente"
```

## Condição de parada

O treinamento encerra quando uma época INTEIRA é percorrida sem nenhum erro em nenhuma amostra.

---

*Rede pronta para operação: pesos ficam congelados e são usados para classificar novas amostras.*

# O conceito de época

- **Época = uma apresentação completa de TODO o conjunto de treinamento à rede.**
- Alterar um único peso sináptico já modifica a inclinação e a posição de toda a reta.
- Logo: mesmo que só 1 padrão em 100 apresente erro, não se pode ajustar esse peso e encerrar — é obrigatório rodar novamente TODOS os padrões da época até obter um ciclo completo com erro zero.

# Aspectos práticos do treinamento

## 1 Não separável

Se o problema não for linearmente separável, a rede diverge (oscila). Solução: limitar por um número máximo de épocas.

## 2 Instabilidade

Faixa de separabilidade muito estreita gera instabilidade no treinamento. Solução: reduzir a taxa de aprendizado  $\eta$ .

## 3 Número de épocas

Varia conforme os valores iniciais de  $w$ , a disposição espacial dos dados e o valor de  $\eta$ .

## 4 Solução não única

Existem diversas retas ou hiperplanos capazes de separar as mesmas duas classes.

# 06

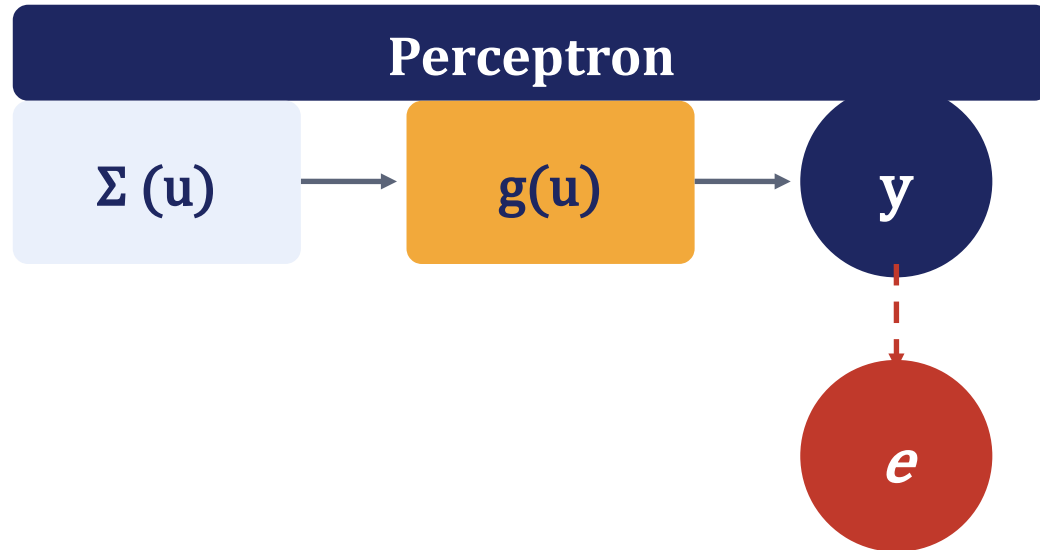
## Perceptron × Adaline

*Duas arquiteturas de camada única, dois métodos de aprendizado*

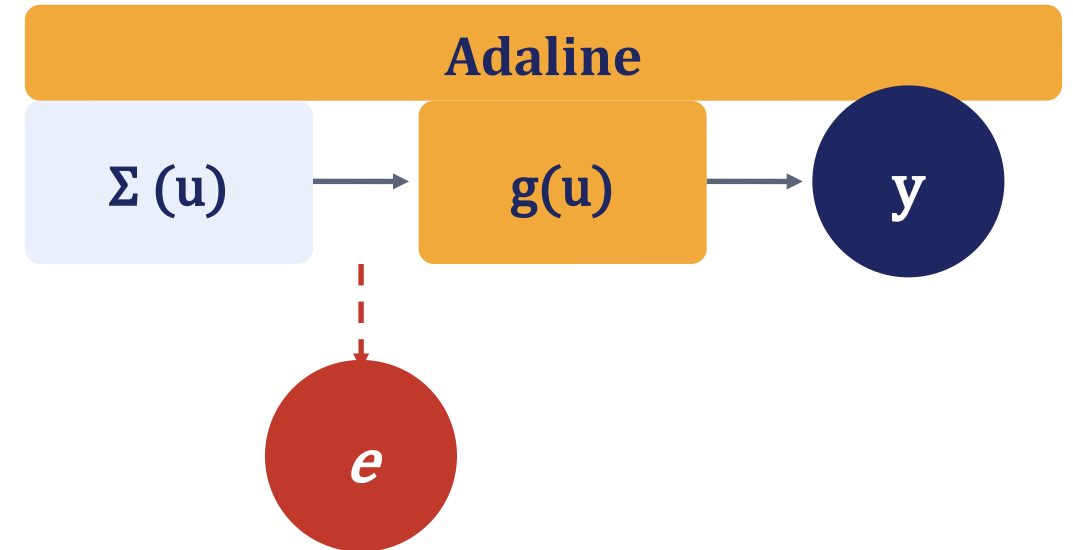
# Apresentando a Adaline

- Desenvolvida por Bernard Widrow e Ted Hoff, em 1960.
- Base para a Regra Delta (regra LMS — Least Mean Squares / mínimos quadrados).
- A Regra Delta foi generalizada na Regra Delta Generalizada: algoritmo fundamental do Backpropagation, usado para treinar o Multilayer Perceptron (PMC).
- Grande impacto prático em processamento de sinais e cancelamento adaptativo de ruídos em transmissões de rádio e TV.

# A diferença fundamental: onde o erro é extraído



$$e = d - y \text{ (após ativação)}$$



$$e = d - u \text{ (antes da ativação)}$$

No Perceptron, o erro é extraído depois do degrau (saída discreta). Na Adaline, o erro é extraído antes — direto do combinador linear contínuo  $u$ . A ativação discreta só entra na fase de operação, para gerar a decisão final.

# Função de custo e Gradiente Descendente

Enquanto o ajuste do Perceptron nasceu de intuição neurobiológica (Hebb), a regra de aprendizado da Adaline é derivada rigorosamente pela Análise Matemática, via Gradiente Descendente.

Erro Quadrático Médio (EQM)

$$E(W) = \frac{1}{2} (d_k - u_k)^2$$

*O fator  $\frac{1}{2}$  é conveniência matemática, para cancelar a constante 2 na diferenciação.*

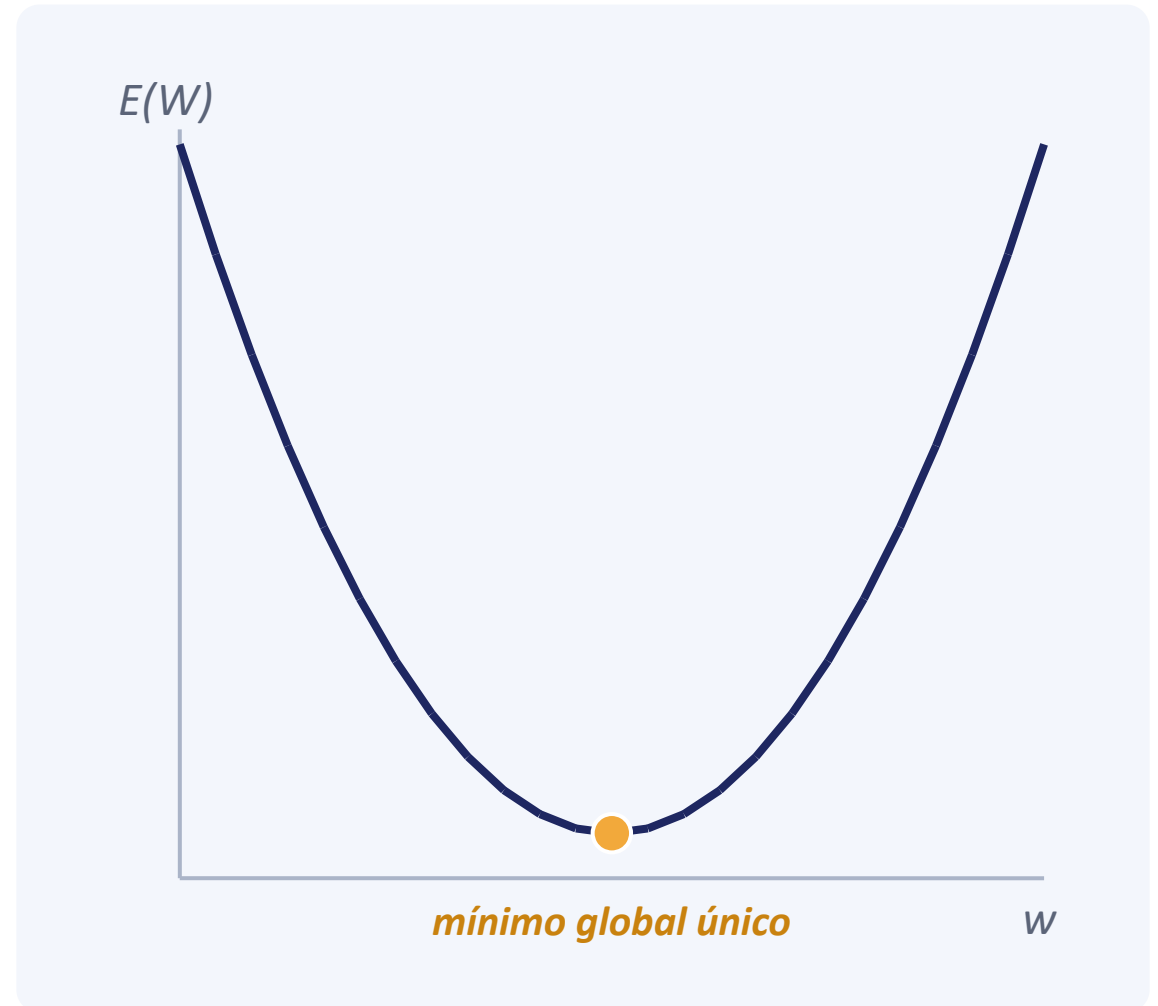
Gradiente e ajuste

$$\nabla E(W) = -(d_k - u_k) \cdot X_k$$

*O gradiente aponta para a máxima subida do erro; caminhamos no sentido OPOSTO para minimizar.*

# Superfície de erro: um único mínimo global

- A função de custo é quadrática em relação aos pesos.
- Sua superfície de erro assume a forma de uma parábola (parabolóide em dimensões superiores).
- **Possui um único ponto de mínimo global — não existem mínimos locais no treinamento da Adaline.**



# Regra Delta (LMS) e critério de parada

$$W(t+1) = W(t) + \eta \cdot (d_k - u_k) \cdot X_k$$

*Equação de atualização contínua — Regra Delta / LMS (Least Mean Squares)*

## Critério de parada infinitesimal ( $\epsilon$ )

Como se trabalha com variáveis reais contínuas e precisão de ponto flutuante, é inviável exigir erro zero absoluto (diferente do Perceptron). O treinamento é interrompido avaliando a variação do EQM entre duas épocas sucessivas:

$$|EQM(atual) - EQM(anterior)| \leq \epsilon \quad (\text{ex.: } \epsilon = 10^{-6} \text{ ou } 10^{-7})$$

# Interpretação geométrica e robustez

- No Perceptron, dependendo da inicialização, a reta pode parar em qualquer lugar da região de separabilidade — inclusive colada às amostras de uma classe.
- Na Adaline, a minimização do EQM força a reta a se posicionar exatamente no centro da região de separabilidade.
- Unicidade: os pesos finais da Adaline são sempre os mesmos para a mesma base de dados, independente da inicialização (que afeta apenas o nº de épocas).
- Robustez: por ficar equidistante das duas classes, a Adaline é muito mais resistente a ruídos nas amostras de teste.

# Quadro comparativo: Perceptron × Adaline

| Característica           | Perceptron                      | Adaline                                        |
|--------------------------|---------------------------------|------------------------------------------------|
| Origem / Autor           | Frank Rosenblatt (1958)         | Widrow & Hoff (1960)                           |
| Extração do erro         | Após a ativação ( $e = d - y$ ) | Antes da ativação ( $e = d - u$ )              |
| Fundamento de ajuste     | Regra de Hebb (neurobiológico)  | Gradiente Descendente (LMS)                    |
| Superfície de erro       | Descontínua / discreta          | Parabólica, mínimo global único                |
| Posição da fronteira     | Qualquer reta válida na região  | Exatamente no centro                           |
| Pesos finais             | Variam com a inicialização      | Únicos para a mesma base                       |
| Problemas não separáveis | Não converge (oscila)           | Converge (minimiza o EQM)                      |
| Critério de parada       | Erro zero na época completa     | $\Delta EQM \leq \varepsilon$ (ex. $10^{-6}$ ) |

# O que levar desta aula



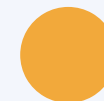
## Estrutura

Um único neurônio, feedforward, sem realimentação.



## Matemática

$u = \sum w_i x_i - \theta$ ;  $y = g(u)$ ;  $\theta$  incorporado como  $w_0$ .



## Geometria

Discriminador linear: reta, plano ou hiperplano.



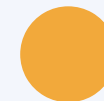
## Convergência

Só converge se as classes forem linearmente separáveis (XOR falha).



## Treinamento

Regra de Hebb:  $w \leftarrow w + \eta(d-y)x$ , por épocas completas.



## Adaline

Erro antes da ativação + Gradiente Descendente  $\rightarrow$  solução única e mais robusta.